

Tests et preuve de programmes

I) Exponentiation rapide

```
def exp_rapide(x, n):
    """ Pour n un entier et x un nombre (entier ou flottant), calcule l'exponentielle
    de x par n en  $O(\log(n))$  opérations.
    """
    resultat = 1
    while n > 0:
        # si n est pair, on factorise un 2 dans le calcul :  $x^n = (x^2)^{n/2}$ 
        if n % 2 == 0:
            x *= x
            n /= 2
        # sinon, on multiplie le résultat par x, pour enlever 1 à n.
        else:
            resultat *= x
            n -= 1
    return resultat
```

Question 1

Proposez un partitionnement de l'espace d'entrée, et des tests aux cas limites, pour la fonction exp_rapide.

Question 2

Donnez un variant de boucle pour la fonction exp_rapide. En déduire sa terminaison.

Question 3

Donnez une pre-condition, un invariant, et une post-condition pour la fonction exp_rapide. En déduire sa correction.

II) Tri par insertion

Je vous donne le code du tri par insertion :

```
def insere(L_triee, x):
    """ Cette fonction prend en argument une liste triée (en ordre croissant)
    d'entiers, et un élément x, et l'insère en place dans la liste. """
    for i in range(len(L_triee)):
        if x < L[i]:
            x, L[i] = L[i], x #inverse x et L[i] si ils sont dans le désordre
    #à la fin, l'élément restant est le plus grand de la liste
    L_triee.append(x)

def tri_insertion(L):
    """ Cette fonction prend en argument une liste d'entier, et renvoie une nouvelle
    liste triée en ordre croissant avec l'algorithme du tri par insertion. """
    L_triee = []
    for i in range(len(L)):
        insere(L_triee, L[i])
    return L_triee
```

Question 4

Proposez des tests aux cas limites, pour :

- a) la fonction `insere`
- b) la fonction `tri_insertion`

Question 5

Donnez une pre-condition, un invariant, et une post-condition pour la fonction `insere`. En déduire sa correction partielle.

Ici, la terminaison des deux fonctions est évidente, car elles ne contiennent ni appels récursifs, ni boucles `while`

Question 6

En supposant la fonction `insere` correcte, donnez une pre-condition, un invariant, et une post-condition pour la fonction `tri_insertion`. En déduire sa correction.

III) Tri rapide

```
def tri_rapide(L):
    """ Pour L une liste, renvoie une nouvelle liste triée par ordre croissant
    contenant les mêmes éléments.
    """
    if (len(L) <= 1):
        return L[:]
    else:
        pivot = L[0]
        #separe la liste en deux : les éléments plus grand et plus petits que le
        pivot
        plus_petits = [el if el <= pivot for el in L[1:]]
        plus_grands = [el if el > pivot for el in L]

        #on trie ces deux listes en appelant récursivement notre fonction tri_rapide
        plus_petits_triee = tri_rapide(plus_petits)
        plus_grands_triee = tri_rapide(plus_grands)

        #enfin, on concatène ces listes pour obtenir le résultat
        return plus_petits_triee + [pivot] + plus_grands_triee
```

Question 7

Les tests de la fonction `tri_insertion` sont-ils aussi valides pour `tri_rapide` ?

Question 8

Donnez un variant pour la fonction `tri_rapide`, c'est-à-dire une quantité qui décroît à chaque appel récursif. En déduire sa terminaison.

Question 9

Montrez la correction de `tri_rapide`.